# Contributions on Schedule-Aware Bundle Routing

Projet de Programmation 2

DAGUE Bastien, AL-FARAJ Samy, JOUINI Yassine, BROCHARD Vincent

L3 Informatique - Faculté des Sciences - Université de Montpellier

2025 - 2026

## Table des matières

**Glossaire et Abréviations** **2**

**1 Introduction** **3**
1.1 A-SABR . . . 3
1.2 Sujet . . . 3

**2 Développements Logiciel : Conception, Modélisation, Implémentation** **4**
2.1 DT-Chat . . . 4
2.1.1 DT-Chat Backend . . . 4
2.1.2 DT-Chat Frontend . . . 5
2.2 Pipeline . . . 6
2.2.1 Première version du fichier : . . . 6
2.2.2 Optimisation du fichier de base : . . . 7
2.2.3 Implémentation de hooks pour la pipeline . . . 7
2.3 Test A-SABR . . . 8
2.3.1 Module : Contact Manager . . . 8
2.3.1.1 Legacy : . . . 8
2.3.1.2 Segmentation : . . . 8
2.3.1.3 PrioritySegmentation : . . . 9
2.3.2 Module : Pathfinding . . . 10
2.3.2.1 Try_make_hop : . . . 10
2.3.2.2 Contact,Hybrid et Node parenting : . . . 10
2.3.3 Module : Routing . . . 11
2.3.3.1 VolCgr . . . 11
2.3.3.2 Spsn . . . 11
2.4 BP-Socket . . . 11

**3 Algorithmes et Structures de Données** **12**
3.1 PrioritySegmentation . . . 12
3.2 Hybrid Parenting . . . 12

**4 Analyse des résultats** **13**
4.1 Benchmark . . . 13

**5 Gestion du Projet** **14**
5.1 Organisation prévue en début de projet . . . 14
5.2 Organisation réelle du travail . . . 14

**6 Bilan et Conclusions** **16**

**7 Annexe de codes** **17**

# Glossaire et Abréviations

**BP Socket**

Bundle Protocole Socket. Type spécifique de socket utilisée dans le cas des DTN [3]. 3, 4

**Bundle**

Equivalent du paquet dans les échanges tradionnels, se définit par sa source, sa ou ses destinations, sa priorité, sa taille et son délai d'expiration.. 10

**CP**

Contact Plan. Fichier de configuration décrivant les fenêtres de communication planifiées entre nœuds : qui communique avec qui, à quelle heure, pendant combien de temps, avec quel débit et quel délai. Lu par la bibliothèque, il est transformé en Multigraph (Multigraph). 3–6

**DTN**

Delay-Tolerant Networking. Réseau tolérant aux délais et aux ruptures de connectivité, où les nœuds ne sont pas forcément connectés en permanence. Les données sont transmises de manière opportuniste via des contacts planifiés.. 3

**Multigraph**

Structure de données centrale du projet représentant le réseau. Contient les nœuds (senders/receivers) et les contacts. Permet plusieurs contacts entre une même paire de nœuds à des moments différents, d'où le préfixe "multi". 2, 10, 18

**PRELOAD**

Mécanisme du Dynamic Loader des systèmes UNIX permettant de charger un module dont les fonctions seront utilisées en priorités avant celles de la glibC. 3

**SABR**

Schedule-Aware Bundle Routing. Algorithme de routage pour réseaux DTN qui utilise des métriques de distance (temps d'arrivée, nombre de hops) pour trouver les meilleures routes entre nœuds. [1]. 3, 10, 11

# 1 Introduction

## 1.1 A-SABR

Notre projet de programmation porte sur la librairie ASABR, portée par Mr Olivier De Jonckère du LIRMM.

Le réseau de communication spacial repose sur le Delay-Tolerant Networking (DTN). Avec ce type de protocole, le routage s'effecture par des algorithmes de type Schedule-Aware Bundle Routing (SABR).

La librairie A-SABR implémente donc ce type d'algorithme en proposant une grande modularité et une bonne performance.[1] C'est également pour suivre ce choix de performance que celle-ci est programmé en Rust, language connue pour son compilateur stricte et sa fine gestion de la mémoire.

## 1.2 Sujet

L'intérêt pour ce sujet a été immédiat au sein de notre groupe, motivé par la perspective de contribuer à un projet Open Source d'envergure réelle. Ce cadre de travail nous a offert l'opportunité de nous initier au langage Rust, dont la montée en puissance dans l'industrie logicielle souligne l'importance actuelle. Pour plusieurs membres de l'équipe aspirant à se spécialiser en informatique système (bas niveau) ou en réseaux, ce projet constituait une application concrète et exigeante de nos ambitions professionnelles.

Le sujet présentait plusieurs point à aborder :

- Portage au no-std
- Divers tâche de DevOps (Tests, Pipeline)
- Création d'un nouvel onglet dans l'interface graphique DT-Chat
- Intégration dans le logiciel Hardy

Au final, tout à été un peu chamboulé, le no-std à été abbandonnée, l'intégration à Hardy jugée peut intéressante.

Nous avons surtout compris qu'ici on ne dépendait pas seulement de nous mais qu'il s'agissait d'un immense travail d'équipe. On commence une tâche, on la trouve accompli, on lance une Pull Request, on discute, on modifie, tous ça prend un certain temps et nécessite beaucoup de vas et viens.

Arrivé à la fin du projet voici les réels objectifs accompli :

**Interface graphique DT-Chat :** Développement d'un volet « Options » dans l'interface graphique DT-Chat : Ce module permet désormais de modifier dynamiquement, au cours de l'exécution, l'algorithme de routage utilisé ou le Contact Plan (CP) chargé.

**Automatisation de test unitaires :** Mise en place d'une pipeline CI/CD robuste pour assurer la pérennité de la bibliothèque. Ce travail a été complété par la mise en place de plusieurs tests unitaires couvrant les modules principaux du projet. La pipeline teste la compilation, l'execution des tests ainsi que la rédaction de la coverage sous forme d'HTML

**Expérimentation Système (BP Socket) :** Pendant le dernier mois du projet, nous avons travaillé sur Bundle Protocole Socket (BP Socket).

BP Socket telle qu'elle à été développée utilise un module kernel nécessitant les droit root pour être utilisée. L'objectif est de permettre l'usage de BP Socket sur des systèmes embarqués contraints où les privilèges « root » ne sont pas disponibles. L'expérimentation du mécanisme LD_PRELOAD (PRELOAD) vise à intercepter les appels système pour s'affranchir de la dépendance au module noyau actuel.

Ce dernier objectif est une total découverte de ce mécanisme offert par le Dynamic Loader et toute avancée même non complète intéresse Mr De Jonckère.

La compilation de la librairie et de l'interface graphique nécessitant divers paquet Linux, notre travail à été réalisé soit par WSL, soit directement depuis une distribution Linux.

# 2 Développements Logiciel : Conception, Modélisation, Implémentation

## 2.1 DT-Chat

DT-Chat est une application graphique codée en Rust permettant d'illustrer une communication DTN. Elle supporte UDP,TCP et BP Socket.

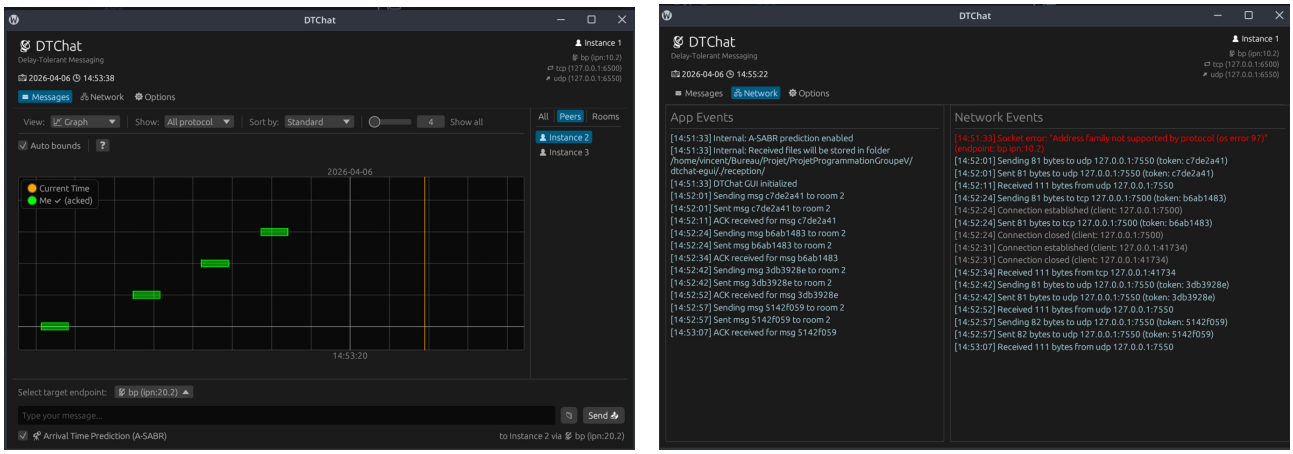



FIGURE 1 – Onglet Message (gauche) et Events (droite) d'un terminal DT-Chat

A gauche nous pouvons voir l'onglet principal, nous voyons un graphique des différents message envoyée, un rectangle vert signifie un message envoyée et reçu avec confirmation. En bas il y a la possibilité de choisir entre UDP,TCP et BP Socket. Egalement, il y a la possibilité de cocher la méthode A-SABR, ceci permet qu'a l'envoie d'un message l'application prédit et indique le moment où le message devrait être reçu par le recepteur.

A droite nous voyons l'onglet Event, celui ci recapitule de manière textuelle toute interaction de l'interface. A gauche le recapitulatif de l'interaction logicielle et à droite celle du reseau, on voit bien les différents message envoyée et reçu.

Sur cette exemple, il y a eu 3 envois du même terminal, 1 UDP, 1 TCP, 1 BP Socket. BP Socket dans le network Event est marqué comme UDP car il s'agit d'une variante d'UDP.

L'objectif sur cette interface graphique est de créer un nouvel onglet Options permettant de changer à la volée de CP et/ou d'algorithme. Cela nécessite donc un travail à la fois coté Backend et FrontEnd

Maintenant, détaillons le travail effectuée sur chacun des parties de cette interface.

### 2.1.1 Backend

(Code en annexe 7)

Premièrement, on créer une fonction update tiré de la fonction start de l'interface, celle-ci prend en argument un str et un chemin de fichier. Il faut aussi modifier la fonction de lancement de l'interface, qui ici se nomme try_init pour qu'elle prenne également un str et un chemin de fichier, on récupère également les valeurs de départ qu'on met dans l'appel de la fonction. try_init à aussi été modifie pour retourner le nombre de node et de contact trouvée dans le CP chargé.

La fonction update prend donc en entrée un str qui correspond au nom de l'algorithme utilisée, et un Path sous forme de string qui correspondra au chemin jusqu'au CP.

Elle va ensuite faire un match sur `"PredictionConfig::try_init(path.clone(), algo)"` qui retournera donc un Result, pour empêcher un crash total de l'interface au moindre soucis.

Si Ok alors on met à jour la configuration qu'on envoie au Frontent et on l'affiche dans l'App Event, si Err, on retourne l'erreur et on l'affiche dans l'App Event.

Etant basé sur A-SABR, il faut régulièrement faire de petite modif dûe à des refactorisation de fonction ou de l'ajout de Result remplaçant des Options dans la librairie.

Toute ces modifications sur la partie Backend ont fait l'objet de 2 Pull request acceptées de Vincent pour 52 lignes de code ajoutée (PR #6 et #7).

### 2.1.2 Frontend

(Code en annexe 7)

L'interface DT Chat peut être vue comme un modèle MVC en programmation Web, le Modèle serait le Backend, la Vue toute les interactions possibles avec l'interface, et le contrôleur le code qui résulte de ces interactions.

Ici le but est donc d'afficher ce fameux onglet Options et d'envoyer les différents ordre au Backend suivant les actions faite.

Le raccourci ci-dessus permet d'accéder au code la vue Options, il se résume simplement en un bouton qui ouvre un explorateur pour choisir son CP et une ComboBox pour le choix d'algorithme, dès qu'une action est réalisé pour un ou pour l'autre, l'interface se met directement à jour et affiche les nouveaux paramètres chargées dans l'App Event.

Une spécificité existe pour le choix d'algorithme, certain algorithme sont conditionnées à l'activation ou non de "features", des fonctionnalités qu'on choisit d'activer ou non lors du lancement de l'interface, on peut voir dans le code que la ComboBox s'adapte aux choix de features choisies lors de la compilation.

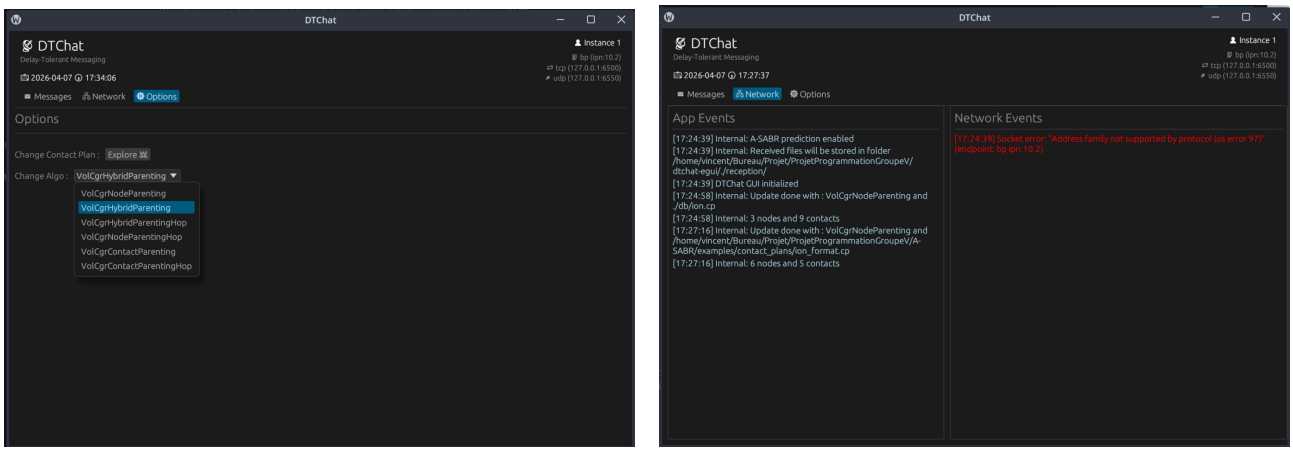
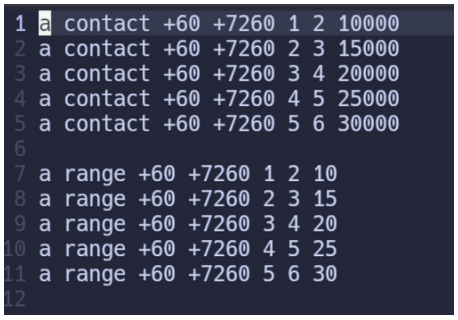



FIGURE 2 – Onglet App Event mis à jour (gauche) et ComboBox adaptatif (droite) d'un terminal DT-Chat

L'interface graphique à sur cette exemple illustrée été compilé avec la feature `contact_work_area`, donc la ComboBox s'est adaptée. J'ai ensuite sélectionnée l'algorithme `VolCgrNodeParenting` ainsi que chargé le CP `ion_format.cp` et nous voyons bien dans l'App Event que le changement est effectif et que le nombre de node et de contacts à bien été détectée.

Dans les fait, l'algorithme choisit ne change pas réellement ce que l'interface nous montre lors de la réception ou l'envoi de message, il s'agit surtout ici de préparer de possible futur ajoux car le Backend lui comprend totalement les changement effectués.

Pour finir, un soucis à été détecté en ce qui concerne le changement de CP.



```
a contact +60 +7260 1 2 10000
a contact +60 +7260 2 3 15000
a contact +60 +7260 3 4 20000
a contact +60 +7260 4 5 25000
a contact +60 +7260 5 6 30000

a range +60 +7260 1 2 10
a range +60 +7260 2 3 15
a range +60 +7260 3 4 20
a range +60 +7260 4 5 25
a range +60 +7260 5 6 30
```

FIGURE 3 – Contact plan "A-SABR/examples/contact_plans/ion_format.cp"

Ci-dessus, on a le format classique de CP que l'interface comprend correctement, hors il existe plusieurs autre format. De base il n'y avait pas l'affichage du nombre de nodes et de contact détectée lors du chargement d'un contact plan, mais j'ai trouvée étrange que l'appli prennent sans erreurs tous type de fichiers et donc, après investigation et discussion, on à décider de faire afficher le nombre de contact et de noeuds. La plupart des contact plan qui ne sont pas en format "ION" comme ci-dessus ne sont pas du tous compris par l'application. Elle ne crash pas mais affiche simplement 0 contacts et 0 noeuds.

```
# Node entry with no management: node <id> <name>
node 0 node1
node 1 node2
node 2 node3
node 3 node4
node 4 node5
node 5 node6

# Dynamic parsing for contacts a marker should appear before the manager tokens:
# contact <from> <to> <start> <end> <marker> ...
contact 0 1 60 7260 eto 10000 10
contact 1 2 60 7260 evl 15000 15
contact 2 3 60 7260 evl 20000 20
contact 3 4 60 7260 qd 25000 25
contact 4 5 60 7260 qd 30000 30
contact 4 0 60 7260 qd 30000 30

# A segmented contact after the marker, the parser expects a sequence of delay/rate interval
# delay <start> <end> <delay>
# rate <start> <end> <rate>
contact 0 5 60 7260 seg rate 60 3660 10000 rate 3660 7260 15000 delay 60 7260 12

# A-SABR format does not care of whitespaces, including new lines:
contact 1 4 60 7260 seg
delay 60 3660 10
delay 3660 7260 15
rate 60 7260 10000
```

FIGURE 4 – Contact plan "A-SABR/examples/contact_plans/asabr_format_dynamic.cp"

Ci-dessus un exemple de contact plan qu'il ne lit pas du tous.

Pour régler le soucis, il aurait fallu développer tous un parser qui s'adapte au type de CP chargé. Dans les fait c'était réalisable mais cela prenait beaucoup de temps sachant que le coeur du sujet est A-SABR donc on a juste rajouté le compteur de noeuds et de contacts.

Toutes ces modifications sur le FrontEnd ont fait l'objet d'une Pull Request de Vincent acceptée pour 145 lignes de codes ajoutée (PR `#13`).

## 2.2 Pipeline CI/CD

### 2.2.1 Première version du fichier :

L'une des tâches principales de notre projet était d'apporter des pipelines d'automatisation pour faciliter le travail futur de l'équipe. Pour ce faire, il a fallu déterminer ce dont le projet avait besoin et comment le mettre en place grâce aux outils propres au langage Rust. Après en avoir discuté avec Mr De Jonckère, nous avons décidé de travailler sur 4 aspects principaux : le format du code, la validité, l'optimisation et son bon fonctionnement grâce aux tests. Commençons par détailler la première version de notre travail (à noter que le fichier présenté ensuite est commun à `A-SABR`, `dtchat-egui` et `dtchat-backend`). (Code en annexe 7)

Ce fichier ci.yml crée dans le dossier `.github/workflows` se présente de la manière suivante :

— Les déclencheurs : comme nous pouvons le voir, il est précisé que le fichier doit se lancer dans les branches `main`, `develop` et toutes les branches commençant par `feature/` afin d'offrir à notre équipe localement le travail de cette Pipeline.

— L'environnement : afin d'effectuer notre travail, il nous faut travailler dans un environnement Linux. Nous utilisons donc `ubuntu-latest` pour la richesse de ses outils. Cependant, un problème est rapidement apparu : utiliser cet environnement tel quel fait appel à bien trop de ressources, ainsi nous avons décidé d'utiliser le container `rust-slim` car c'est un bon compromis entre légèreté et la présence des outils nécessaires.

— La préparation du travail : il nous faut ensuite configurer tous les outils que nous utiliserons ensuite.

  Tout d'abord, `actions/checkout@v4` s'occupe de récupérer le code source du projet, ensuite pour avoir la version stable du compilateur Rust nous utilisons `dtolnay/rust-toolchain@stable` avec deux outils supplémentaires indispensables : `rustfmt` et `clippy` dont nous reparlerons. Enfin, pour avoir de meilleurs performances et un temps d'exécution réduit nous utilisons `swatinem/rust-cache@v2`, cela nous permet sauvegarder les dépendances téléchargées pour les réutiliser ensuite.

— Le travail effectué : maintenant que tout est prêt, nous pouvons utiliser nos commandes pour faire le travail attendu.

  Pour commencer, `cargo fmt -all - -check` est l'outil qui vérifie si le code respecte bien le format standard Rust, nous remarquons que l'appel de cette commande est suivi par `continue-on-error: true` cela veut dire que si une erreur de format est détectée elle est signalée mais n'arrête pas le workflow.

  Le second outil utilisé est `cargo clippy`. Il nous permet de vérifier beaucoup de choses cruciales pour le projet : la bonne compilation du code et si malgré une compilation réussie il reste des erreurs de logique. Ces erreurs peuvent être de différents types : trop d'arguments dans une fonction, code inutile ou encore des erreurs de sécurité. Pour terminer, nous ajoutons `cargo test` pour, comme son nom l'indique, lancer nos tests implémentés.

  On y ajoute les cibles `-lib`, `-bins` et `-tests` afin de tester respectivement la partie bibliothèque du code, les exécutables et les fichiers du dossier `tests/`.

Avec cette première version, nous répondons bien aux problèmes évoqués plus tôt. Cependant, après discussion lors de l'ouverture de la Pull Request, il était clair que ce fichier manquait d'optimisation et n'était pas assez strict (les erreurs

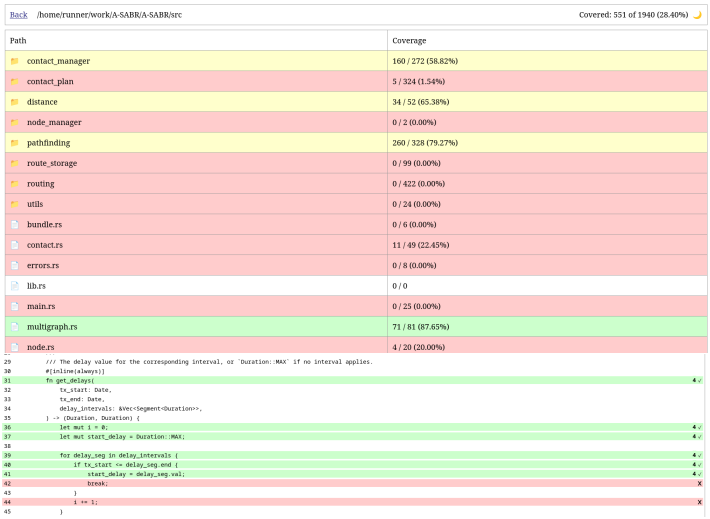
fmt et clippy sont tolérées). Tout est rassemblé en un seul job rendant plus difficile le diagnostic d'une potentielle erreur et aucun commentaire ne permet aux personnes ne connaissant pas cette syntaxe de comprendre le fonctionnement du fichier.

### 2.2.2 Optimisation du fichier de base :

Après avoir reçu les premiers retours, voici ce qui en est ressorti : la manque de clarté du fichier, le fait d'autoriser certaines erreurs, un temps d'installation des dépendances pouvant être raccourcis et la possibilité de rajouter un job de coverage. Afin de répondre aux attentes, voici comment le fichier a été corrigé (à noter qu'ici les modifications ne concernent que le fichier lié à `A-SABR`, nous reparlerons de `dtchat` dans la section suivante) : (Code en annexe 7)

- Grâce à la partie `concurrency`, nous permettons à Github d'arrêter l'exécution en cours de la Pipeline si un nouveau commit est poussé afin que seul ce dernier profite des vérifications. Cela permet de gagner un peu de temps si on pousse un commit avec une erreur puis qu'on la corrige directement dans le commit suivant.
- Pour répondre aux problèmes de clarté, nous avons extrait les différentes vérifications en 4 jobs : `fmt`, `clippy`, `tests` et `coverage`. De plus, grâce à la balise `name`, nous ajoutons un titre à chaque nouvelle commande afin de permettre à d'autres de mieux comprendre le travail effectué.
- Il fallait également rendre la Pipeline moins permissive, nous avons donc retiré la mention `continue-on-error: true` dans le job `fmt` afin de faire en sorte que les erreurs de format deviennent bloquantes. Concernant clippy, on lui ajoute les cibles `-all-targets`, `-all-features` et `- -D warnings`. Ces ajouts permettent respectivement de lancer clippy sur l'intégralité du code, d'utiliser toutes les features disponibles et de rendre les avertissements bloquants.
- Concernant la partie test, nous avons juste ajouté la cible `-verbose` à `cargo test` afin d'avoir des messages d'erreurs plus détaillés.
- Enfin, nous avons ajouté un job de coverage. Le but de ce travail est de générer un fichier html qui permettra de voir où en est la couverture de code globale du projet. Pour ce faire, nous utilisons l'outil tarpaulin. Le fichier généré est mis dans les artefacts GitHub et sauvegardé pendant 7 jours. Ce job étant le plus demandant en ressources, nous avons décidé avec Mr De Jonckère de ne l'exécuter que lorsqu'il est appelé sur la branche main grâce à `if: github.ref == 'refs/github/main` Dans les captures d'écran suivantes, nous pouvons voir le poucentage de couverture pour chaque dossier et fichier. Quand nous cliquons sur un fichier, les lignes couvertes apparaissent en vertes et celles non couvertes en rouge afin de mieux guider le développeur implémentant les tests.

Il est intéressant de remarquer également les lignes du type `if: ${{ !env.ACT }}`. ACT est un outil installé localement permettant de lancer le fichier ci.yml directement sur notre machine sans l'envoyer vers GitHub permettant de garder un historique plus propre sur le dépôt.



Back /home/runner/work/A-SABR/A-SABR/src — Covered: 551 of 1940 (28.40%)

| Path | Coverage |
|---|---|
| contact_manager | 160 / 272 (58.82%) |
| contact_plan | 5 / 324 (1.54%) |
| distance | 34 / 52 (65.38%) |
| node_manager | 0 / 2 (0.00%) |
| pathfinding | 260 / 328 (79.27%) |
| route_storage | 0 / 99 (0.00%) |
| routing | 0 / 422 (0.00%) |
| utils | 0 / 24 (0.00%) |
| bundle.rs | 0 / 6 (0.00%) |
| contact.rs | 11 / 49 (22.45%) |
| errors.rs | 0 / 8 (0.00%) |
| lib.rs | 0 / 0 |
| main.rs | 0 / 25 (0.00%) |
| multigraph.rs | 71 / 81 (87.65%) |
| node.rs | 4 / 20 (20.00%) |

```
29          /// The delay value for the corresponding interval, or `Duration::MAX` if no interval applies.
30          #[inline(always)]
31          fn get_delays(                                                                          4 ✓
32              tx_start: Date,
33              tx_end: Date,
34              delay_intervals: &Vec<Segment<Duration>>,
35          ) -> (Duration, Duration) {
36              let mut i = 0;                                                                      4 ✓
37              let mut start_delay = Duration::MAX;                                                4 ✓
38
39              for delay_seg in delay_intervals {                                                  4 ✓
40                  if tx_start <= delay_seg.end {                                                  4 ✓
41                      start_delay = delay_seg.val;                                                4 ✓
42                      break;                                                                      ✗
43                  }
44                  i += 1;                                                                         ✗
45              }
```

FIGURE 5 – Captures d'écrans du fichier de coverage géneré

Grâce à ces ajouts, nous avons implémenté un travail de meilleure qualité tout en ajoutant des fonctionnalités non prévues au début. Cependant, la bonne utilisation de ces fonctionnalités dépend d'un envoi vers le serveur GitHub. Nous avons bien essayé de contourner ce problème via ACT mais cet outil est soumis au même temps d'exécution que celui sur le serveur ce qui peut devenir très chronophage lorsqu'il faut le tester plusieurs fois.

### 2.2.3 Implémentation de hooks pour la pipeline

Lors de la suite de nos discussions, Mr De Jonckère nous a suggéré d'utiliser le principe de pre-commit grâce aux hooks. C'était à ce moment du travail une notion que nous ne maîtrisions pas, ce qui nous a pris un temps de recherche

pour bien adapter cette solution au projet. Le principe est simple, comme le nom "pre-commit" l'indique l'analyse interviendra juste avant de préparer le commit du travail effectué et pourra le bloquer s'il ne respecte pas certaines règles. On procèdera donc à une analyse locale avant envoi puis une analyse après envoi. Pour cela, il nous a fallu créer un fichier `.pre-commit-config.yaml` et modifier notre `ci.yml`. Voyons plus en détail ces nouveaux fichiers en commençant par le fichier de pre-commit : (Code du fichier de pre-commit en annexe 7)

— Pour travailler nous utiliserons 2 repositories, le premier utilise des outils disponibles via un dépôt GitHub, le second utilise notre système local.

  Le premier repo, nous permet l'utilisation de 4 outils : `trailing-whitespace` qui supprime les grands espaces dans le code, `end-of-file-fixer` met à la fin de tous les fichiers un unique saut de ligne, `check-yaml` vérifie la bonne syntaxe des fichiers .yml et .yaml et `check-added-large-files` empêche de faire des commits trop lourds (ajout accidental d'un dossier de fichiers compilés).

  Le second repo est utilisé afin d'utiliser les outils Rust avec lesquels nous avons travaillé jusqu'ici à savoir cargo fmt et cargo clippy, l'appel des ces outils reste le même que précedemment. Nous pouvons relever un détail qui a son importance qui est la ligne `language: system`. En effet, cette ligne précise que le fichier devra aller chercher les outils Rust dans la toolchain locale de l'utilisateur, nous pouvions également demander d'aller chercher les outils dans un dépôt en ligne cependant nous avons considéré que nos machines sont régulièrement mises à jour donc les problèmes de différences de versions ne gêneront pas et que ce serait plus simple à maintenir si le dépôt venait à ne pas plus être mis à jour.

Pour profiter des fonctionnalités de ce fichier, on peut utiliser la commande `pre-commit run -all-files` qui lancera tous les jobs de pre-commit sur l'ensemble des fichiers.

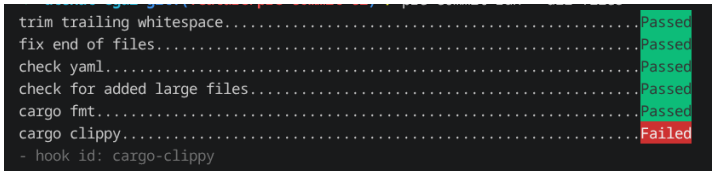



FIGURE 6 – Exemple d'un résultat de lancement de la commande avec détection d'erreurs

Mais le plus intéressant avec cet outil est la possibilité de bloquer un commit défectueux. Après avoir installé pre-commit sur sa machine, les jobs se lanceront sur les fichiers qui ont été staged au moment de taper `git commit ...` Ainsi, grâce à cela il est impossible d'envoyer sur le dépôt un commit qui ne respecte pas les règles établies.

Concernant le fichier `ci.yml` (Code du fichier ci.yml en annexe 7), un changement majeur est à relever : les jobs lancés localement par le fichier de pre commit sont rappelés par la ligne `uses: pre-commit/action@v3.0.1` dans un job nommé "lint". Grâce à cela on assure une stabilité plus importante du code et une meilleure optimisation future car le fichier `ci.yml` utilise le contenu du fichier de pre-commit.

Le reste du fichier présente peu de changements par rapport à la version précédente, nous lançons les tests dans un job dédié suivi du job de couverture des tests.

Pour conclure cette partie, nous avons su au fur et à mesure de l'avancée du travail relever nos erreurs et les corriger tout en prenant en compte les remarques et besoins des autres collaborateurs du projet ce qui nous a permis de découvrir de nouvelles notions et de consolider nos connaissances déjà existantes dans ce domaine.

Notre travail a finalement été accepté dans le dépôt de Monsieur De Jonckère (PR#30) et peut maintenant servir à toute l'équipe.

## 2.3 Test A-SABR

### 2.3.1 Contact Manager

Expliquez bien ici ce qu'est un contact manager

#### 2.3.1.1 Legacy :

Partie de Yassine et ou Samy expliquer aussi en quoi ils sont dépréciés et pas utilisée

#### 2.3.1.2 Segmentation :

Ici ta partie Yassine, montre en quoi c'est une evolution des contacts legacy

#### 2.3.1.3 PrioritySegmentation :

Le module `PrioritySegmentation` abrégé en `pseg`, quant à lui, reprend exactement le même principe que le module `Segmentation` mais ajoute, comme son nom l'indique, une gestion de la priorité des paquets. Le principe central de ce module est le suivant : un paquet de plus haute priorité peut remplacer un segment occupé par un paquet de plus petite priorité. L'intérêt de cette approche est de pouvoir mettre en place une hiérarchie dans l'importance des paquets envoyés, dans le cas du routage spatial, on préfèrera faire passer un message d'urgence avant un retour d'analyse quelconque, c'est ce que permet `pseg`. Ci-dessous un schéma d'explication du fonctionnement de `pseg`.

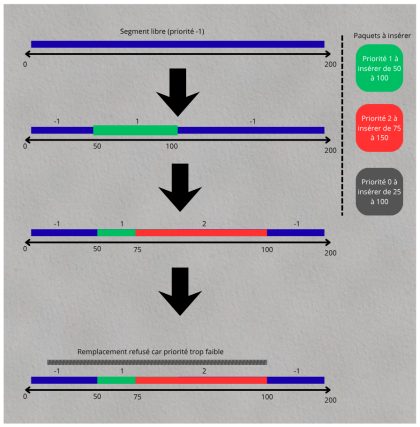



FIGURE 7 – Schéma de fonctionnement de PrioritySegmentation

On observe tout d'abord que les segments libres sont notés d'une priorité -1 tandis que les paquets peuvent avoir une priorité allant de 0 à 2. Un paquet de plus haute priorité ne remplace pas forcément tout le paquet de priorité inférieure, on peut voir qu'au moment de l'insertion du paquet de priorité 2 en rouge seule la partie dont il avait besoin a été allouée et n'a pas remplacé l'entièreté du segment occupé par le paquet de priorité 1.

Le fonctionnement de ce module est assez similaire à celui de `Segmentation` expliqué précedemment. Nous retrouvons les segments de débit et de délai pour construire l'univers dans lequel nous travaillerons. Cependant nous n'utilisons pas ici des intervalles libres mais des segments appelés `bookings` qui seront initialisés avec une date de début et de fin, une priorité -1 pour indiquer qu'ils sont libres.

Nous retrouvons le modèle des deux fonctions centrales : `dry_run_tx` pour la simulation et `schedule_tx` pour l'application effective des modifications. L'analyse algorithmique du module sera traitée dans la partie 3.

Dans le cadre du projet, nous avons apporté certaines modifications au module. Une procédure permettant de fusionner les `bookings` adjacents de même priorité a été implémenté et une couverture de tests importante a été faite. De plus, durant cette phase de tests, un bug a été descelé et résolu : il existait un cas où l'insertion échouait quand elle devait être faite à la fin du segment alors qu'elle devait réussir. Détaillons maintenant le travail implémenté pour créer nos tests. Pour avoir des tests satisfaisants, nous devions vérifier l'égalité des retours des fonctions `dry_run_tx` et `schedule_tx` et couvrir tous les différents scénarios qu'il était possible de rencontrer. Au début, des tests simples ont été faits : une fonction de test par scénario. Cette approche fonctionnait et offrait une bonne couverture mais elle pouvait être très largement simplifié pour les futurs utilisateurs. Après en avoir discuté avec Monsieur De Jonckère, nous avons mis au point une fonction centrale `start_test` qui s'occupe de construire l'environnement et de lancer le test associé et des fonctions qui appelaient notre `start_test` avec des valeurs différentes, expliquons le principe de cette fonction : (Code de la fonction `start_test` en annexe 7)

- Le principe global : la fonction prend 3 paramètres qui sont `input` une liste d'objets de type InputSeg afin de construire l'environnement de base, `output` qui est une liste de bookings de type OutpuSeg représentant les bookings attendus par le retour de la fonction et `requests` une liste de tuples respectant la norme suivante : en premier, le paquet de type `Bundle` à insérer, ensuite le moment où il doit être inséré (de type f64) et un booléen mis à `true` si on s'attend à ce que l'inserion réussise et `false` sinon.
- La construction de l'environnement : quand la fonction est appelé elle va utiliser la liste `input` pour construire l'environnement de test. Par exemple, avec un input de type let input = vec ![InputSeg : :Delay(0.0, 200.0, 4.0),InputSeg : :Rate(0.0, 200.0, 100.0),] ; la fonction va faire un matching sur les différents champs pour construire les segments utiles, elle crée le segment de délai avec les valeurs 0, 200 et 4 et le segment de débit avec les valeurs 0, 200 et 100. Si il n'y a pas de champ `booking` dans l'input alors on part d'une liste de `bookings` vide sinon on peut partir d'une liste avec des `bookings` déjà existants.
- Les tests effectués : maintenant que l'environnement est crée, nous pouvons commencer les insertions de paquets.

Pour chaque paquet dans la liste `requests` on va récupérer les résultats de `dry_run_tx` et de `schedule_tx` afin de tester s'ils sont égaux.

Ensuite, on vérifie que l'insertion du paquet est bien en accord avec le booléen représentant la réussite de l'insertion, si le test renvoie une erreur c'est que le paquet a été inséré alors qu'on s'attendait à un échec ou le contraire.

Enfin, on construit la sortie réelle de la fonction et on vérifie qu'elle est bien équivalente à la sortie attendue représentée par le champ `output`.

La force de cette fonction est qu'elle permet de faire 3 tests communs en un seul appel pour différents scénarios. De plus, si des collaborateurs souhaitent écrire de nouveaux tests, ils n'auront besoin que de construire les valeurs attendues sans avoir à réecrire à chaque fois les mêmes principes de tests que les précédents.

### 2.3.2 Pathfinding

Le rôle de ce module est de calculer, à partir d'un Bundle (Bundle) et d'un instant donnée, la meilleurs route vers une destination en tenant compte des contraites temporelles de chaque contact. Il expose un Trait, `PathFinding`, implémentée par trois variantes : Node, Contact et Hybride, détaillée ci-après, ainsi qu'un sous module `limiting_contact` qui permet de supprimer temporairement le contact le plus limitant d'une route pour forcer la découverte de routes alternatives.

Chaque variante est instanciée avec une métrique de distance, soit **Hop** qui minimise le nombre de saut, soit **SABR** qui minimise le temps d'arrivée.

#### 2.3.2.1 Try_make_hop :

Ici ta partie Samy

#### 2.3.2.2 Contact,Hybrid et Node parenting :

Contact,Hybride et Node sont 3 variantes de l'algorithme de Dijkstra qui sont utilisés pour le routage :

Node est le plus simple, il explore le Multigraph crée noeud par noeud, et ne maintient qu'une seule route optimal. Il trouve le chemin optimal selon la métrique choisie mais s'arrête dès que la destination est atteinte en mode Path, contrairement aux autres variantes qui continuent d'explorer.

Contact explore lui le Multigraph par contact plutôt que par noeud, il va utiliser une "Work Area", permettant d'explorer les contacts directement sans passer par la logique nœud par nœud, ce qui peut trouver des routes avec moins de saut dans certains cas.

Hybrid est la plus complète mais également la plus complexe, il va maintenir une liste de potentiel route optimal tous le long du parcours, puis, grâce au fonction "can_retain" et "must_prune", va décider laquelle garder.

Chacunes de ces variantes possédent les 3 mêmes macros :
- Path : Chemin simple vers une destination, s'arrête dès qu'elle est atteinte.
- PathExcl : Si l'on veut exclure un noeud précis.
- TreeExcl : Explore l'entièreté du Multigraph avec exclusion.

Les tests sonts simples, on créer plusieurs Multigraph auquel on applique chacunes des macros, on utilise le manager le plus simple possible pour ne pas rendre le test trop lourd, on applique en dernier la méthode Hop (le moins de saut) ou la méthode SABR (le plus rapide) pour avoir le plus de données de test possible.

Les résultats attendues sont le nombre de saut réalisés, et le temps d'arrivée. Pour les cas d'exclusion, on donne comme objectif par exemple le noeud 3 en excluant le noeud 2 et l'on doit avoir en retour un échec du au noeud exclus.

Dans un soucis de lisibilité et de maintienabilité du code, toute les fonctions de création d'objets (noeud,contact,bundle,multigraph) sont réunis dans un fichiers à part (test_helpers). De cette manière, seul les tests sont présents dans leur fichiers cibles.

Prenons un test en particulier et décryptons le : 7 (Cliquer sur le numéro pour être envoyé à l'annexe correspondant)

Nous avons donc une première fonction qui va créer le Multigraph auquel on va applique nos tests, le multigraph créer possèdents 5 noeuds qui ont un manager neutre (qui ne fait rien) pour faire au plus simple et de 5 contacts.

Exemple : le premier contact émet du noeud 0 au noeud 1, début du contact à 0 et fin à 10, débit de 1 unité/seconde et aucun délai de propagation.

Ensuite nous créeons un Bundle à destination du noeud 4, sans priorité et sans taille précise.

On contruit les deux chemin, un avec la méthode SABR, l'autre avec la méthode Hop, puis nous lui appliquons notre fonction de test.

— Hop : On attend 3 sauts en 50 unité de temps.
— SABR : On attend 4 sauts en 50 unité de temps.

Le fait que le nombre de saut diverge entre Hop et SABR permet une plus grand couverture pour detecter une casse de la librarie.

Cette partie à fait l'objet de 2 Pull Request acceptée de Vincent pour 629 lignes rajoutée correspondant aux tests dans les 3 fichiers + la création du test_helpers du module Pathfinding. (PR `#38` et `#44`)

#### 2.3.3 Routing

Si on à le temps de PR ou d'avoir quelque chose de bien.

##### 2.3.3.1 VolCgr

##### 2.3.3.2 Spsn

### 2.4 BP-Socket

# 3 Algorithmes et Structures de Données

## 3.1 PrioritySegmentation

Jte laisse ça samy, tu détaille exactement ce que fait l'algo tu peux faire des schéma complexité algo et temps tous le bazar tous les types de données que t'utilise

## 3.2 Hybrid Parenting

# 4 Analyse des résultats

## 4.1 Benchmark

# 5 Gestion du Projet

## 5.1 Organisation prévue en début de projet

Lors du début de notre travail, il était prévu de travailler sur ces 3 tâches :
— Les tests et pipelines
— La modification de l'interface graphique `dtchat`
— L'implémentation d'A-SABR dans Hardy
Il était prévu de suivre le calendrier suivant.

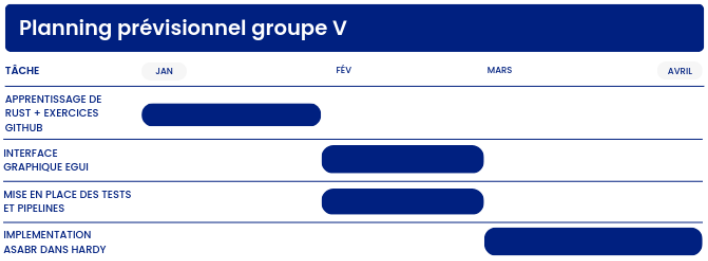



FIGURE 8 – Diagramme de Gantt prévisionnel

Concernant la structure de notre GitHub, comme nous devions travailler sur plusieurs dépôts, nous avons fait le choix de créer un dépôt principal avec à l'intérieur des sous-modules pointant vers les dépôts sur lesquels nous avons travaillé en laissant à la racine tout ce qui concerne le côté scolaire du projet notamment le dossier pour le rapport.

Quant à notre manière de travailler, il était prévu de suivre la structure bien connue Gitflow.

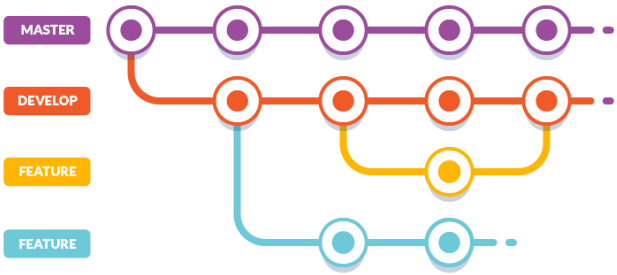



FIGURE 9 – Structure Gitflow

## 5.2 Organisation réelle du travail

Après avoir débuté notre travail, nous avons rapidement compris que les choix d'organisation que nous avons fait allaient devoir rapidement être adaptés afin d'être plus efficace.

Pour commencer, nous voulions suivre une structure Gitflow, donc travailler dans des branches feature/ puis mettre en commun dans la branche develop pour ensuite envoyer une version stable vers notre main puis vers le dépôt de Monsieur De Jonckère. Mais en commençant notamment nos tests, nous nous sommes vite rendus compte que la mise en commun dans develop n'était pas nécessaire puisque nos tâches ne dépendaient pas de la réussite des autres Cette approche était même plus compliquée à manipuler pour notre encadrant : à chaque Pull Request de notre main vers le sien, il aurait dû vérifier un très grand nombre de modifications en une seule fois.

La solution trouvée a donc été la suivante : notre main suivra toujours la branche main du dépôt d'origine, à chaque nouvel ajout nous créeons une branche depuis le main, sur cette branche on effectue un `git rebase main` pour récupérer le travail le plus récent et nous commençons à travailler. Quand le travail est jugé satisfaisant, nous ouvrons une Pull Request vers le main du dépôt d'origine afin d'avoir des retours. Après la prise en compte des suggestions, la Pull Request est acceptée et notre travail est merge dans le dépôt d'origine.

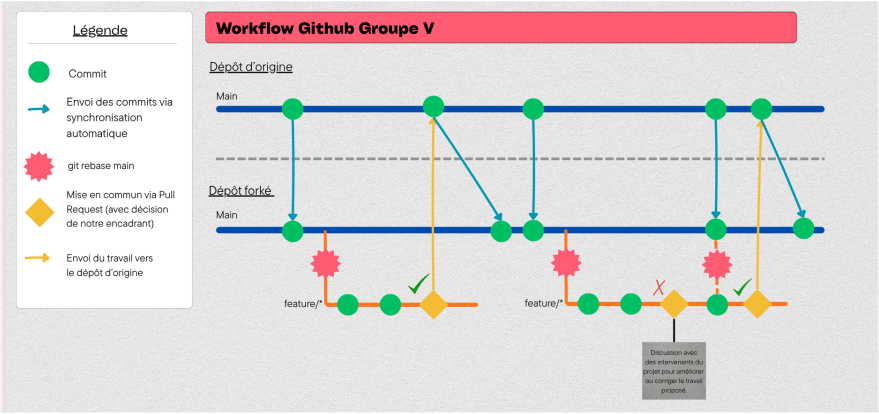



FIGURE 10 – Structure suivie par le groupe

Concernant les tâches effectivement réalisées, la majorité des objectifs prévus ont été atteints : modification de l'interface `dtchat`, ajout de nombreux tests et pipeline performante. Certains changements de tâches ont quand même eu lieu au cours du projet : le travail d'intégration dans Hardy a été mis de côté pour laisser la place au travail de BP-Socket jugé plus intéressant pour nous et pour Monsieur De Jonckère.

Même si l'objectif BP-Socket n'a été atteint que partiellement par manque de temps, le début de travail déjà fait pourrait amener à quelque chose de plus élaboré lors de potentielles nouvelles contributions en dehors du cadre de projet de programmation.

Pour conclure, ce projet a été des plus formateurs pour nos qualités de gestion de projet. Nous avons dû confronter nos connaissances académiques à celles du monde professionnel afin d'en tirer les conclusions qui ont le mieux fonctionné pour nous. Même si une petite sensation de désordre s'est fait ressentir en début de parcours, elle nous a permis de devenir par la suite plus performants avec les outils Git et de faire preuve d'une meilleure flexibilité face à un environnement inconnu.

# 6 Bilan et Conclusions

Bien que les orientations initiales aient évolué pour s'adapter aux contraintes techniques rencontrées, les jalons majeurs de développement ont été atteints.

Notre effort s'est concentré sur la fiabilisation de l'existant et l'extension des capacités de la bibliothèque.

— Mise en place d'une pipeline CI/CD spécifique pour la librairie.
— Mises en places de nombreux test unitaires pour la librairie.
— Refactorisation et ajout d'un onglet pour l'interface graphique DT-Chat.
— Correction de bug dans la librairie.
— Ajout d'un système d'agrégat dans PrioritySegmentation.
— Expérimentation d'interception d'appel système via LD_PRELOAD pour BP-Socket.

| **Composant** | **Nombre de tests** | **Cible technique** |
|---|---|---|
| Module Legacy | 39 | Stabilité des fonctions historiques |
| Parenting (Node, Contact, Hybrid) | 18 | Logiques de routage et filiation |
| Fonction `Try_Make_Hop` | 12 | Précision du calcul de prochain saut |
| Segmentation (Standard & Priority) | 10 | Intégrité et agrégation des données |
| **Total au 13 Avril** | **79** | **Couverture des modules les plus bas** |

TABLE 1 – Répartition de l'effort de test par module.

Je noterai ici la coverage quand la PR de Samy sera acceptée.

La pertinence de notre suite de tests a été confirmée lors de récente modifications de la librairie par Mr De Jonckère. Nos tests ont agi comme un filet de sécurité critique, permettant d'identifier immédiatement les régressions et facilitant ainsi une évolution rapide du code et évitant la casse de la librairie.

Pour résumé l'état d'esprit du projet, on débarque sur quelque chose de déjà en partie construit, dans un language moderne, nouveau et d'autant plus Open Source.

Cela à nécessité un certain temps d'adaptation et de modifications de nos habitudes scolaires pour produire quelque chose de sérieux et professionnel. C'était extraordinaire à faire, apprendre le système de Pull Request, apprendre à produire du code fiable et maintenable, manier Git dans les détails et surtout avoir son nom dans les contributeurs d'un projet important.

# 7 Annexe de codes

2.1.1 (Cliquer sur le chapitre pour revenir au texte correspondant)

La fonction `update` est tiré de DTChat Backend "`src/dtchat.rs`" ligne 218
La fonction `try_init` est tiré de DTChat Backend "`src/prediction.rs`" ligne 30
`pred_res` est tiré de DTChat Backend "`src/config/mod.rs`" ligne 83

```
    pub fn update(&mut self, path:String, algo: &str){
        match PredictionConfig::try_init(path.clone(), algo){
            Ok(update_config) => {
                self.a_sabr = ASabrInitState::Enabled(update_config);
                self.notify_observers(ChatAppEvent::Info(format!("Update done with : {path} and  {algo}
            }
            Err(error) => {
                self.a_sabr = ASabrInitState::Error(error.to_string());
                self.notify_observers(ChatAppEvent::Info("Update failed".to_string()));
            }
        }
    }

pub fn try_init(cp_path: String, algo : &str) -> io::Result<Self> {
        let cp = IONContactPlan::parse::<NoManagement, EVLManager>(&cp_path)?;

        let nodes_length = cp.nodes.len();
        let contacts_length = cp.contacts.len();

        let node_index_map: HashMap<String, NodeID> = cp.nodes
            .iter()
            .enumerate()
            .map(|(index, node)| (node.get_node_name().to_string(), index as NodeID))
            .collect();

        let router_box = build_generic_router::<NoManagement, EVLManager>(
            algo,
            cp,
            None,
        ).expect("Failed to build router");

        let router: Box<dyn Router<NoManagement, EVLManager> + Send + Sync> =
            unsafe { std::mem::transmute(router_box) };
        // in seconds
        let cp_start_time = DTChatTime::now().timestamp_millis() as f64 / 1000.0;

        Ok(PredictionConfig {
            ion_to_node_id: node_index_map,
            router,
            cp_start_time,
            nodes_length,
            contacts_length,
        })
    }


let pred_res = PredictionConfig::try_init(cp_path_unwrapped,"VolCgrHybridParenting");
```

2.3.2.2 (Cliquer sur le chapitre pour revenir au texte correspondant)

La fonction de création d'un Multigraph `exemple_2_graph` provient de A-SABR `"src/pathfinding/test_helpers.rs"` ligne 229.

La fonction `test_exemple_2` qui permet de tester le Multigraph précedemment créer grâce à la fonction ci dessus est présente dans les trois fichiers `Node,Contact et Hybrid Parenting` dans A-SABR `"src/pathfinding"`.

```rust
pub(crate) fn exemple_2_graph()
-> Result<Rc<RefCell<Multigraph<NoManagement, EVLManager>>>, ASABRError> {
    Ok(Rc::new(RefCell::new(Multigraph::new(ContactPlan::new(
        vec![
            make_node(0, "source", NoManagement {}),
            make_node(1, "from_C0", NoManagement {}),
            make_node(2, "from_C2_C1", NoManagement {}),
            make_node(3, "from_C3", NoManagement {}),
            make_node(4, "from_C4", NoManagement {}),
        ],
        vec![
            make_contact::<NoManagement>(0, 1, 0.0, 10.0, 1.0, 0.0),
            make_contact::<NoManagement>(0, 2, 25.0, 35.0, 1.0, 0.0),
            make_contact::<NoManagement>(1, 2, 10.0, 20.0, 1.0, 0.0),
            make_contact::<NoManagement>(2, 3, 20.0, 40.0, 1.0, 0.0),
            make_contact::<NoManagement>(3, 4, 50.0, 60.0, 1.0, 0.0),
        ],
        None,
    )?)?)))
}

fn test_exemple_2() -> Result<(), ASABRError> {
        let mg = exemple_2_graph()?;

        let mut algo_hop = NodeParentingPath::<NoManagement, EVLManager, Hop>::new(mg.clone());
        let mut algo_sabr = NodeParentingPath::<NoManagement, EVLManager, SABR>::new(mg.clone());

        let bundle = make_bundle(4, 0, 0.0, 1000.0);

        let res_hop = algo_hop
            .get_next(0.0, 0, &bundle, &[][..])
            .expect("Routing Failed !");

        assert_time_hop(&res_hop, 4, 50.0, 3, "Hop");

        let res_sabr = algo_sabr
            .get_next(0.0, 0, &bundle, &[][..])
            .expect("Routing Failed !");

        assert_time_hop(&res_sabr, 4, 50.0, 4, "SABR");

        Ok(())
    }
}
```

(Code du chapitre 2.1.2)

Entièreté du code de la vue Options de DT Chat Egui "src/options_view.rs"

```rust
    use crate::main_view::MirroredData;
use eframe::egui;
use egui_file_dialog::FileDialog;


pub struct OptionsView{
    pub selected_algo: String,
    pub selected_cp_path: String,
    file_dialog: FileDialog,
}

impl OptionsView{
    pub fn new() -> Self {
        Self {
            selected_algo: "VolCgrHybridParenting".to_string(),
            selected_cp_path: "./db/ion.cp".to_string(),
            file_dialog: FileDialog::new(),
        }
    }
    pub fn show(&mut self,ui: &mut egui::Ui, _data: &mut MirroredData) -> Option<(String,String)>{
        self.file_dialog.update(ui.ctx());
        let mut update = false;
        ui.vertical(|ui| {
            ui.add_space(5.0);
            ui.heading("Options");
            ui.add_space(5.0);
            ui.separator();
            ui.add_space(15.0);

            //Contact Plan
            ui.horizontal(|ui| {
                ui.label("Change Contact Plan :");
                if ui.button("Explore ").clicked() {
                    self.file_dialog.pick_file();
                    }
            });

            if let Some(path) = self.file_dialog.take_picked(){
                self.selected_cp_path = path.display().to_string();
                update = true;
            }

            ui.add_space(10.0);

            //Algorithme
            ui.horizontal(|ui| {
                ui.label("Change Algo :");
                let old_algo = self.selected_algo.clone();

                let mut algos: Vec<&str> = vec![
                    "VolCgrNodeParenting",
                    "VolCgrHybridParenting",
                    "VolCgrHybridParentingHop",
                    "VolCgrNodeParentingHop",
                ];

```

```
                #[cfg(feature = "contact_work_area")]
                algos.extend([
                    "VolCgrContactParenting",
                    "VolCgrContactParentingHop",
                ]);

                #[cfg(feature = "contact_suppression")]
                algos.extend([
                    "CgrFirstEndingHybridParentingHop",
                    "CgrFirstEndingHybridParenting",
                    "CgrFirstEndingNodeParentingHop",
                    "CgrFirstEndingNodeParenting",
                ]);

                #[cfg(all(feature = "contact_work_area", feature = "contact_suppression"))]
                algos.extend([
                    "CgrFirstEndingContactParenting",
                    "CgrFirstEndingContactParentingHop",
                ]);

                #[cfg(all(feature = "contact_suppression", feature = "first_depleted"))]
                algos.extend([
                    "CgrFirstDepletedHybridParentingHop",
                    "CgrFirstDepletedHybridParenting",
                    "CgrFirstDepletedNodeParentingHop",
                    "CgrFirstDepletedNodeParenting",
                ]);

                #[cfg(all(feature = "contact_work_area", feature = "contact_suppression", feature = "f
                algos.extend([
                    "CgrFirstDepletedContactParentingHop",
                    "CgrFirstDepletedContactParenting",
                ]);

                egui::ComboBox::from_id_salt("algo_selector")
                    .selected_text(&self.selected_algo)
                    .show_ui(ui, |ui| {
                        for algo in algos{
                            ui.selectable_value(&mut self.selected_algo, algo.to_string(), algo);
                        }
                    });
                if old_algo != self.selected_algo{
                    update = true;
                }
            });
        });
        if update{
            Some((self.selected_cp_path.clone(),self.selected_algo.clone()))
        } else{
            None
        }
    }
}
```

2.2.3 (Cliquer sur le chapitre pour revenir au texte correspondant)
Code complet de la première version de notre fichier ci.yml :

```yaml
    name: CI

on:
  push:
    branches: ["develop","main","feature/*"]

jobs:
  test:
    runs-on: ubuntu-latest
    container:
      image: rust:slim

    steps:
      - uses: actions/checkout@v4
      - uses: dtolnay/rust-toolchain@stable
        with:
          components: rustfmt, clippy
      - uses: swatinem/rust-cache@v2

      - run: cargo fmt --all -- --check
        continue-on-error: true
      - run: cargo clippy
      - run: cargo test --lib --bins --tests
```

2.2.3 (Cliquer sur le chapitre pour revenir au texte correspondant)
Code complet de la seconde version de notre fichier ci.yml :

```yaml
name: A-SABR CI

on:
  push:
    branches: ["**"]
  pull_request:
    branches: ["**"]

concurrency:
  group: ci-${{ github.ref }}
  cancel-in-progress: true

jobs:
  fmt:
    name: Check formatting with Rustfmt
    runs-on: ubuntu-latest
    steps:
      - name: Check out the repository
        uses: actions/checkout@v4

      - name: Set up Rust toolchain
        uses: dtolnay/rust-toolchain@stable
        with:
          components: rustfmt


      - run: cargo fmt --all -- --check

  clippy:
    name: Linting with Clippy
    runs-on: ubuntu-latest
    steps:
      - name: Check out the repository
        uses: actions/checkout@v4

      - name: Set up Rust toolchain
        uses: dtolnay/rust-toolchain@stable
        with:
          components: clippy

      - uses: swatinem/rust-cache@v2
      - run: cargo clippy --all-targets --all-features -- -D warnings

  tests:
    runs-on: ubuntu-latest
    steps:
      - name: Check out the repository
        uses: actions/checkout@v4

      - name: Set up Rust toolchain
        uses: dtolnay/rust-toolchain@stable
      - uses: swatinem/rust-cache@v2
      - run: cargo test --lib --bins --tests --verbose

  coverage:
    name: Code Coverage
    runs-on: ubuntu-latest
```

```yaml
    if: github.ref == 'refs/heads/main'
    needs: tests
    steps:
        - uses: actions/checkout@v4
        - uses: dtolnay/rust-toolchain@stable
        - uses: swatinem/rust-cache@v2

        - name: Install tarpaulin
          if: ${{ !env.ACT }}
          uses: taiki-e/install-action@cargo-tarpaulin

        - name: Generate coverage report
          if: ${{ !env.ACT }}
          run: cargo tarpaulin --ignore-tests --out Html --out Xml

        - name: Upload report
          if: ${{ !env.ACT }}
          uses: actions/upload-artifact@v4
          with:
            name: code-coverage-report
            path: tarpaulin-report.html
            retention-days: 7
```

2.2.3 (Cliquer sur le chapitre pour revenir au texte correspondant) Code complet du fichier `.pre-commit-config.yaml`.

```yaml
repos:
  - repo: https://github.com/pre-commit/pre-commit-hooks
    rev: v4.5.0
    hooks:
      - id: trailing-whitespace
      - id: end-of-file-fixer
      - id: check-yaml
      - id: check-added-large-files

  - repo: local
    hooks:
      - id: cargo-fmt
        name: cargo fmt
        entry: cargo fmt --all -- --check
        language: system
        types: [rust]
        pass_filenames: false

      - id: cargo-clippy
        name: cargo clippy
        entry: cargo clippy --all-targets --all-features -- -D warnings
        language: system
        types: [rust]
        pass_filenames: false
```

2.2.3 Code complet de la version finale du fichier `ci.yml`.

```yaml
name: A-SABR CI

on:
  push:
    branches: ["**"]
  pull_request:
    branches: ["**"]

concurrency:
  group: ci-${{ github.ref }}
  cancel-in-progress: true

jobs:
  lint:
    name: Lint and Style (Pre-commit)
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: dtolnay/rust-toolchain@stable
        with:
          components: clippy, rustfmt
      - uses: swatinem/rust-cache@v2
      - uses: pre-commit/action@v3.0.1

  tests:
    name: Run Tests
    needs: lint
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: dtolnay/rust-toolchain@stable
      - uses: swatinem/rust-cache@v2
      - name: Run Tests
        run: cargo test --all-features --all-targets

  coverage:
    name: Code Coverage
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main'
    needs: tests
    steps:
      - uses: actions/checkout@v4
      - uses: dtolnay/rust-toolchain@stable
      - uses: swatinem/rust-cache@v2
      - uses: taiki-e/install-action@cargo-tarpaulin
      - run: cargo tarpaulin --ignore-tests --out Html
      - uses: actions/upload-artifact@v4
        with:
          name: code-coverage-report
          path: tarpaulin-report.html
          retention-days: 7
```

2.3.1.3 Code de la fonction `start_test`.

```rust

#[derive(Debug, PartialEq, Clone)]
enum InputSeg {
    Delay(Date, Date, Duration),
    Rate(Date, Date, DataRate),
    Booking(Date, Date, Priority),
}

#[derive(Debug, PartialEq, Clone)]
enum OutputSeg {
    Booking(Date, Date, Priority),
}
#[track_caller]
fn start_test(
    input: Vec<InputSeg>,
    output: Vec<OutputSeg>,
    requests: Vec<(Bundle, f64, bool)>,
) {
    let contact_info = ContactInfo::new(0, 1, 0.0, 200.0);
    let mut delay_segments: Vec<Segment<Date>> = Vec::new();
    let mut rate_segments: Vec<Segment<DataRate>> = Vec::new();
    let mut actual_output = Vec::new();
    let mut initial_bookings: Vec<Segment<Priority>> = Vec::new();

    for seg in input {
        match seg {
            InputSeg::Delay(start, end, val) => {
                delay_segments.push(Segment { start, end, val })
            }
            InputSeg::Rate(start, end, val) => rate_segments.push(Segment { start, end, val }),
            InputSeg::Booking(start, end, priority) => initial_bookings.push(Segment {
                start,
                end,
                val: priority,
            }),
        }
    }
    let mut manager = PSegmentationManager::new(rate_segments, delay_segments);
    manager.try_init(&contact_info);
    if !initial_bookings.is_empty() {
        manager.booking = initial_bookings;
    }

    for (i, (bundle, at_time, expect_success)) in requests.iter().enumerate() {
        let dry_run_res = manager.dry_run_tx(&contact_info, *at_time, bundle);
        let schedule_tx_res = manager.schedule_tx(&contact_info, *at_time, bundle);

        assert_eq!(
            dry_run_res, schedule_tx_res,
            "TEST N°{} FAILED: dry_run and schedule_tx doesn't match.\n",
            i
        );
        assert_eq!(
            schedule_tx_res.is_some(),
            *expect_success,
            "TEST N°{} FAILED: expected: {} actual: {}",
            i,
            expect_success,
```

```
                schedule_tx_res.is_some()
            );
        }

        //Building actual output

        for seg in &manager.booking {
            actual_output.push(OutputSeg::Booking(seg.start, seg.end, seg.val));
        }
        assert_eq!(
            actual_output, output,
            "TEST FAILED: Actual output is not the one expected."
        );
    }
```

# Références


[1] J. A. Fraire, O. De Jonckère, and S. Burleigh *Routing in the space internet : A contact graph routing tutorial.*, Journal of Network and Computer Applications, 174, 102884.

[2] O. De Jonckère, L. Ma, and J. A. Fraire. *A-SABR : The Adaptive Library for Schedule-Aware Bundle Routing.*, WiSEE 2025 - IEEE International Conference on Wireless for Space and Extreme Environments, Oct 2025, Halifax, Canada.

[3] Sylvain Pierrot, Olivier de Jonckère, Scott Burleigh, Charley Geoffroy. *Design and Implementation of a Native Socket Interface for the Bundle Protocol.* , WiSEE 2025 - 13th Annual IEEE International Conference on Wireless for Space and Extreme Environments, Oct 2025, Halifax, Canada.